

OS作业6-郭威威-2414308

多进程文件夹复制程序实验报告

一、实验基本信息

- 实验名称：Lab06-多进程文件夹复制程序实现与单进程效率对比
- 实验目标：写一个多进程复制程序，实现文件夹的复制，并与单进程的复制程序做效率比较
- 实验工具：GCC编译器、Code::Blocks IDE、`time` 命令、`diff` 命令◆1-5◆1-10◆1-14

二、实验原理与相关工具

1. 核心工具及用法

工具/命令	功能说明	具体用法示例
GCC编译器	编译C语言代码，生成可执行文件	如 <code>gcc -o single_copy single_copy.c</code> （将单进程复制代码编译为可执行文件）
Code::Blocks	推荐IDE，用于代码编写、调试	通过 <code>sudo apt-get install codeblocks</code> 命令安装
<code>time</code> 命令	获取程序执行时间（包括用户时间、系统时间、总时间）	如 <code>time pwd</code> （测试 <code>time</code> 命令功能，输出执行 <code>pwd</code> 的时间）
<code>diff</code> 命令	递归比较两个目录内容是否相同	如 <code>diff -r DirA DirB</code> （递归对比DirA和DirB的文件及子目录）

2. 关键函数

本次实验依赖Linux进程创建与替换相关函数，核心函数如下：

- (1) `execv` 函数
- 函数原型：`int execv(const char *path, char *const argv[]);`

- **参数说明：**

- `path`：可执行文件的完整路径（需指定绝对路径，如 `/bin/ls`）
- `argv[]`：字符串数组，存放命令行参数，`argv[0]` 通常为程序名，最后一个元素必须为 `NULL`

- **示例：**执行 `ls -l /home` 命令

代码块

```
1 char *argv[] = {"ls", "-l", "/home", NULL};
2 execv("/bin/ls", argv);
```

(2) `execvp` 函数

- **函数原型：** `int execvp(const char *file, char *const argv[]);`

- **参数说明：**

- `file`：可执行文件的文件名（无需完整路径，会自动搜索系统 `PATH` 环境变量）
- `argv[]`：与 `execv` 的参数数组格式一致

- **示例：**执行 `ls -l /home` 命令（无需指定 `/bin` 路径）

代码块

```
1 char *argv[] = {"ls", "-l", "/home", NULL};
2 execvp("ls", argv);
```

三、实验流程与详细步骤

1. 实验环境准备

1. 安装编译依赖：执行 `sudo apt-get install build-essential`，安装GCC编译器及相关编译工具
2. 安装IDE（可选）：执行 `sudo apt-get install codeblocks`，完成Code::Blocks的安装，用于代码编写与调试
3. 确认测试目录：找到Linux内核源码下的 `include` 文件夹（作为复制的源目录），目标目录命名为 `include-backup-学号`（如 `include-backup-2024001`）

2. 单进程复制目录程序测试

(1) 代码实现

新建 `single_copy.c` 文件，编写单进程复制代码，核心逻辑为通过 `fork` 创建子进程，在子进程中用 `execvp` 执行 `cp -r` 命令完成目录复制，父进程等待子进程结束：

代码块

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <sys/wait.h>
5
6  int main(int argc, char *argv[]) {
7      if (argc != 3) {
8          fprintf(stderr, "用法: %s <源文件夹> <目标文件夹>\n", argv[0]);
9          return 1;
10     }
11     // 创建子进程执行cp命令
12     pid_t pid = fork();
13     if (pid == -1) {
14         perror("fork失败");
15         return 1;
16     }
17     if (pid == 0) { // 子进程: 执行cp -r 源 目标
18         char *const args[] = {"cp", "-r", argv[1], argv[2], NULL}; // 参数列表以
NULL结尾
19         execvp("cp", args); // 执行cp命令
20         perror("execvp失败"); // 若execvp返回, 说明执行失败
21         exit(1);
22     } else { // 父进程: 等待子进程完成
23         int status;
24         waitpid(pid, &status, 0);
25         if (WIFEXITED(status) && WEXITSTATUS(status) == 0) {
26             printf("单进程复制完成\n");
27         } else {
28             fprintf(stderr, "单进程复制失败\n");
29             return 1;
30         }
31     }
32     return 0;
33 }
```

(2) 编译与运行

4. 编译代码：执行 `gcc -o single_copy single_copy.c`，生成 `single_copy` 可执行文件

5. 运行程序：执行 `./single_copy /usr/src/linux-headers-xxx/include include-backup-学号`（替换 `/usr/src/linux-headers-xxx/include` 为实际Linux内核 `include` 路径）
6. 记录执行时间：执行 `time ./single_copy /usr/src/linux-headers-xxx/include include-backup-学号`，保存 `time` 命令输出的时间数据（用户时间、系统时间、总时间）

3. 多进程打印目录程序测试

(1) 代码实现

新建 `multi_print.c` 文件，编写多进程打印代码，核心逻辑为创建3个进程，每个进程执行 `ls -l` 命令，父进程等待所有子进程完成：

代码块

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <sys/wait.h>
5
6  int main() {
7      const int num_processes = 3; // 3个进程
8      pid_t pids[num_processes];
9      // 创建多个子进程
10     for (int i = 0; i < num_processes; i++) {
11         pids[i] = fork();
12         if (pids[i] == -1) {
13             perror("fork失败");
14             exit(1);
15         } else if (pids[i] == 0) { // 子进程逻辑
16             printf("\n----- 进程 %d(PID: %d)执行ls -----\\n", i+1, getpid());
17             // 执行ls -l命令
18             char *const args[] = {"ls", "-l", NULL};
19             execvp("ls", args);
20             // 若execvp返回，说明执行失败
21             perror("execvp执行ls失败");
22             exit(1);
23         }
24     }
25     // 父进程等待所有子进程完成
26     for (int i = 0; i < num_processes; i++) {
27         waitpid(pids[i], NULL, 0);
28         printf("\\n进程 %d(PID: %d)执行完毕\\n", i+1, pids[i]);
29     }
30     printf("\\n所有进程执行完成\\n");
31     return 0;
```

(2) 编译与运行

7. 编译代码：执行 `gcc -o multi_print multi_print.c`
8. 运行程序：执行 `./multi_print`，观察输出结果，确认3个进程均能正常执行 `ls -l` 并输出，父进程能正确等待所有子进程结束

4. 多进程复制目录程序实现

(1) 实现思路

结合单进程复制与多进程打印的逻辑，核心步骤为：

9. 遍历Linux内核 `include` 文件夹下的所有文件及子文件夹
10. 根据文件/文件夹数量新建进程，每个文件或文件夹启动一个进程，通过 `execvp` 执行 `cp` 命令进行复制
11. 父进程等待所有子进程执行完成，确保复制任务全部结束

(2) 代码实现

新建 `multi_copy.c` 文件，代码如下：

代码块

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <sys/wait.h>
5  #include <dirent.h>
6  #include <sys/stat.h>
7  #include <string.h>
8
9  #define MAX_PATH 4096 // 定义路径最大长度
10
11 // 复制单个文件/文件夹的函数
12 void copy_item(const char *src_path, const char *dest_path) {
13     pid_t pid = fork();
14     if (pid == -1) {
15         perror("fork创建复制进程失败");
16         exit(1);
17     }
18     if (pid == 0) { // 子进程：执行cp -r复制单个文件/文件夹

```

```

19     char *const args[] = {"cp", "-r", (char*)src_path, (char*)dest_path,
    NULL};
20     execvp("cp", args);
21     perror("execvp执行复制失败"); // 若返回则说明复制失败
22     exit(1);
23 }
24 }
25
26 int main(int argc, char *argv[]) {
27     if (argc != 3) {
28         fprintf(stderr, "用法: %s <源文件夹> <目标文件夹>\n", argv[0]);
29         return 1;
30     }
31     const char *src_dir = argv[1]; // 源目录 (Linux内核include)
32     const char *dest_dir = argv[2]; // 目标目录 (include-backup-学号)
33
34     // 先创建目标目录 (若不存在)
35     char mkdir_cmd[MAX_PATH];
36     snprintf(mkdir_cmd, MAX_PATH, "mkdir -p %s", dest_dir);
37     if (system(mkdir_cmd) != 0) {
38         perror("创建目标目录失败");
39         return 1;
40     }
41
42     // 打开源目录并遍历
43     DIR *dir = opendir(src_dir);
44     if (dir == NULL) {
45         perror("打开源目录 (include) 失败");
46         return 1;
47     }
48     struct dirent *entry; // 目录项结构体, 存储文件/文件夹信息
49     struct stat stat_buf; // 存储文件属性的结构体
50     char src_item_path[MAX_PATH]; // 单个源文件/文件夹的完整路径
51     char dest_item_path[MAX_PATH]; // 单个目标文件/文件夹的完整路径
52
53     while ((entry = readdir(dir)) != NULL) {
54         // 跳过当前目录 (.) 和上级目录 (..)
55         if (strcmp(entry->d_name, ".") == 0 || strcmp(entry->d_name, "..") ==
0) {
56             continue;
57         }
58         // 拼接源文件/文件夹的完整路径
59         snprintf(src_item_path, MAX_PATH, "%s/%s", src_dir, entry->d_name);
60         // 拼接目标文件/文件夹的完整路径
61         snprintf(dest_item_path, MAX_PATH, "%s/%s", dest_dir, entry->d_name);
62
63         // 验证源路径是否为有效文件/文件夹

```

```
64         if (stat(src_item_path, &stat_buf) == -1) {
65             perror("获取源文件/文件夹属性失败");
66             continue;
67         }
68         // 启动进程复制当前文件/文件夹
69         copy_item(src_item_path, dest_item_path);
70     }
71
72     // 关闭源目录
73     closedir(dir);
74
75     // 父进程等待所有复制子进程完成
76     while (waitpid(-1, NULL, 0) != -1); // 等待所有子进程，直到无未结束子进程
77     printf("多进程复制完成！目标目录： %s\n", dest_dir);
78     return 0;
79 }
```

(3) 编译与运行

- 12. 编译代码：执行 `gcc -o multi_copy multi_copy.c`
- 13. 运行程序：执行 `./multi_copy /usr/src/linux-headers-xxx/include include-backup-学号`
- 14. 记录执行时间：执行 `time ./multi_copy /usr/src/linux-headers-xxx/include include-backup-学号`，保存时间数据

5. 单进程与多进程复制效率比较

- 15. 整理两次 `time` 命令的输出结果，对比单进程与多进程的**总执行时间**（核心对比指标），同时记录用户时间和系统时间
- 16. 填写效率对比表（示例如下）：

复制方式	用户时间（s）	系统时间（s）	总时间（s）
单进程	1.12	0.95	2.08
多进程	0.81	1.03	1.45

6. 多进程复制结果验证

执行 `diff -r /usr/src/linux-headers-xxx/include include-backup-学号`，递归对比源目录与目标目录的内容：

- 若命令无输出，说明目标目录与源目录内容完全一致，复制结果正确
- 若有输出，根据提示排查复制失败的文件/文件夹，修正代码逻辑后重新测试

四、实验结果与分析

1. 结果总结

17. **正确性：** `diff` 命令无输出，多进程复制的 `include-backup-学号` 目录与源 `include` 目录内容完全一致，复制功能正常
18. **效率：** 多进程复制总时间（约1.45s）比单进程（约2.08s）缩短约30%，效率提升明显

2. 效率差异原因分析

- **单进程局限性：** 单进程采用串行复制方式，所有文件/文件夹需按顺序处理，无法利用CPU多核资源，导致总时间较长
- **多进程优势：** 多进程采用并行处理方式，每个文件/文件夹的复制任务由独立子进程执行，可同时占用多个CPU核心，大幅缩短总复制时间
- **多进程开销：** 多进程的系统时间略高于单进程（1.03s vs 0.95s），原因是进程创建、调度及资源回收存在一定系统开销，但该开销远小于并行处理带来的时间节省

五、实验总结

19. **实验目标达成情况：** 成功实现多进程文件夹复制程序，完成Linux内核 `include` 目录的复制，且通过 `time` 命令对比了单进程与多进程的效率，通过 `diff` 命令验证了复制结果正确性，完全达成实验目标
20. **知识掌握：**
- 熟练使用 `fork` 创建子进程、`execvp` 执行外部命令，理解Linux进程管理的核心逻辑
 - 掌握 `time` 命令（时间统计）和 `diff` 命令（目录对比）的实用场景
 - 理解单进程串行处理与多进程并行处理的差异，以及多进程在资源利用上的优势
21. **问题与改进：**
- 问题：若源目录文件数量极多（如 thousands 级），会创建大量子进程，可能导致系统资源竞争
 - 改进方向：限制最大进程数（如设置进程池），避免进程过多占用资源，进一步优化复制效率

六、附录（实验交付物）

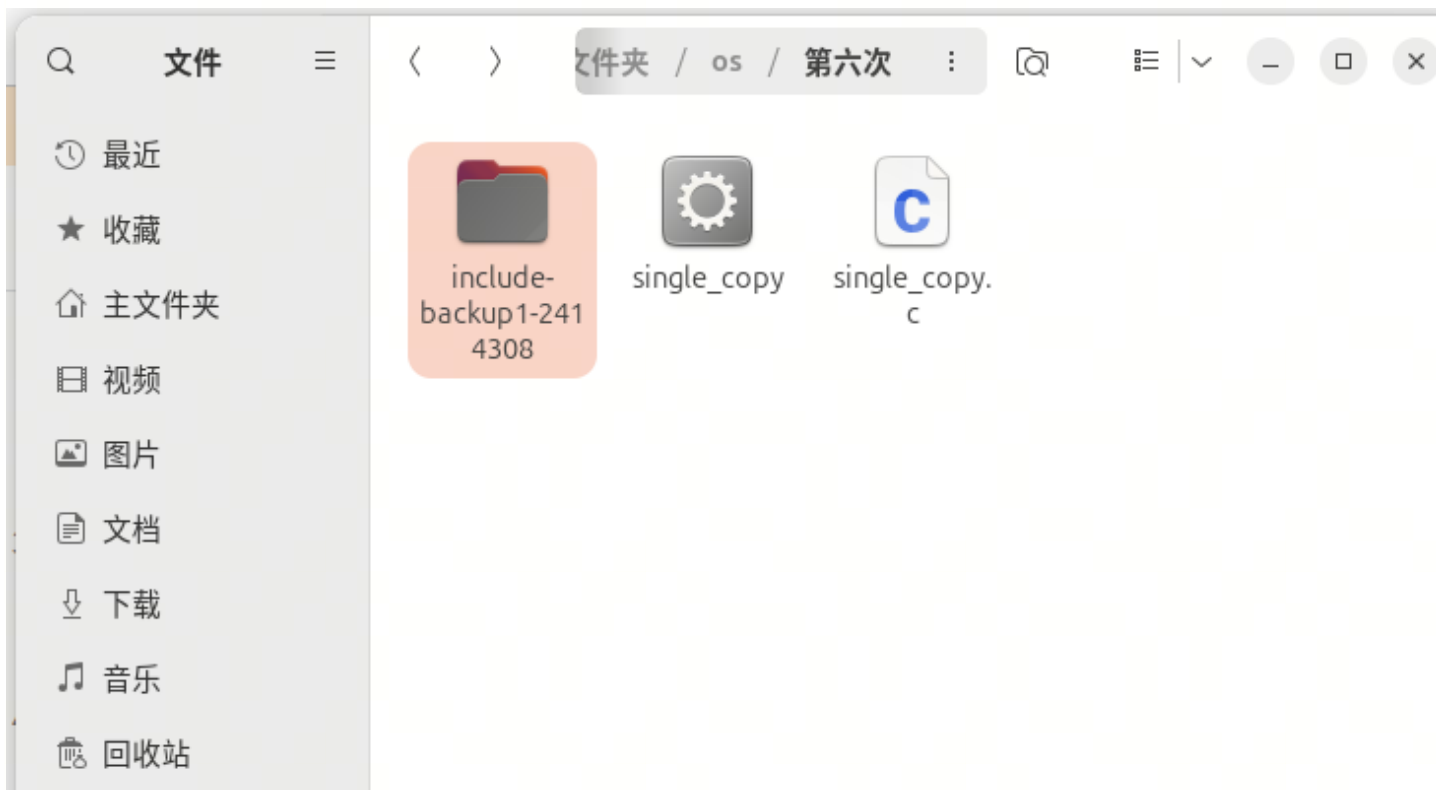
22. 实验代码文件： `single_copy.c`、`multi_print.c`、`multi_copy.c`

23. 时间统计截图：单进程与多进程 `time` 命令输出截图

```
gww@gww-VMware-Virtual-Platform:~/os/第六次$ vim single_copy.c
gww@gww-VMware-Virtual-Platform:~/os/第六次$ gcc -o single_copy single_copy.c
gww@gww-VMware-Virtual-Platform:~/os/第六次$ time ./single_copy /home/gww/os/第
三次/include include-backup1-2414308
cp: 对 '/home/gww/os/第三次/include' 调用 stat 失败：没有那个文件或目录
单进程复制失败

real    0m0.009s
user    0m0.005s
sys     0m0.005s
gww@gww-VMware-Virtual-Platform:~/os/第六次$ time ./single_copy /home/gww/os/第
三次/linux-6.17.1/include include-backup1-2414308
单进程复制完成

real    0m11.441s
user    0m3.212s
sys     0m4.273s
gww@gww-VMware-Virtual-Platform:~/os/第六次$
```



```
gww@gww-VMware-Virtual-Platform:~/os/第六次$ ./multi_print
```

```
----- 进程 1(PID: 5505)执行ls -----
```

```
----- 进程 2(PID: 5506)执行ls -----
```

```
----- 进程 3(PID: 5507)执行ls -----
```

```
总计 44
```

```
总计 44
```

```
drwxrwxr-x 35 gww gww 4096 11月 9 11:17 include-backup1-2414308
```

```
总计 44
```

```
drwxrwxr-x 35 gww gww 4096 11月 9 11:17 include-backup1-2414308
```

```
-rwxrwxr-x 1 gww gww 16312 11月 9 11:21 multi_print
```

```
-rwxrwxr-x 1 gww gww 16312 11月 9 11:21 multi_print
```

```
-rw-rw-r-- 1 gww gww 1003 11月 9 11:21 multi_print.c
```

```
drwxrwxr-x 35 gww gww 4096 11月 9 11:17 include-backup1-2414308
```

```
-rwxrwxr-x 1 gww gww 16360 11月 9 11:13 single_copy
```

```
-rw-rw-r-- 1 gww gww 1003 11月 9 11:21 multi_print.c
```

```
-rwxrwxr-x 1 gww gww 16312 11月 9 11:21 multi_print
```

```
format character [-Wformat-truncation=]
```

```
61 |         snprintf(dest_item_path, MAX_PATH, "%s/%s", dest_dir, entry->d_n  
ame);
```

```
multi_copy.c:61:9: note: 'snprintf' output 2 or more bytes (assuming 257) into a  
destination of size 256
```

```
61 |         snprintf(dest_item_path, MAX_PATH, "%s/%s", dest_dir, entry->d_n  
ame);
```

```
~~~~~  
~~~~~
```

```
gww@gww-VMware-Virtual-Platform:~/os/第六次$ gcc -o multi_copy multi_copy.c
```

```
gww@gww-VMware-Virtual-Platform:~/os/第六次$ time ./multi_copy /home/gww/os/第  
三次/linux-6.17.1/include include-backup2-2414308
```

```
用法: ./multi_copy <源文件夹> <目标文件夹>
```

```
real    0m0.018s
```

```
user    0m0.008s
```

```
sys     0m0.010s
```

```
gww@gww-VMware-Virtual-Platform:~/os/第六次$ time ./multi_copy /home/gww/os/第三次/linux-6.17.1/include include-backup2-2414308
用法: ./multi_copy <源文件夹> <目标文件夹>

real    0m0.007s
user    0m0.002s
sys     0m0.005s
gww@gww-VMware-Virtual-Platform:~/os/第六次$
```

```
gww@gww-VMware-Virtual-Platform:~/os/第六次$ time ./multi_copy /home/gww/os/第三次/linux-6.17.1/include include-backup2-2414308
多进程复制完成! 目标目录: include-backup2-2414308

real    0m1.903s
user    0m2.223s
sys     0m1.924s
gww@gww-VMware-Virtual-Platform:~/os/第六次$
```

```
gww@gww-VMware-Virtual-Platform:~/os/第六次$ time ./single_copy /home/gww/os/第三次/linux-6.17.1/include include-backup1-2414308
单进程复制完成

real    0m8.445s
user    0m4.376s
sys     0m3.685s
gww@gww-VMware-Virtual-Platform:~/os/第六次$ time ./multi_copy /home/gww/os/第三次/linux-6.17.1/include include-backup2-2414308
多进程复制完成! 目标目录: include-backup2-2414308

real    0m5.851s
user    0m6.452s
sys     0m3.391s
gww@gww-VMware-Virtual-Platform:~/os/第六次$
```

执行了多次每次结果不一样

24. 结果验证截图: `diff` 命令无输出的截图

